

DRCGen: A Controllable DRC Violation Case Generator with Cross-PDK Transferability for Rule-Deck and Tool Validation

Abstract

VLSI layout pattern generation plays a crucial role in design for manufacturability (DFM). Patterns that incorporate specific design rule checking (DRC) violations are valuable for applications such as foundry design-rule deck development, EDA tool validation, and generating training data for AI-based DRC research. However, existing research primarily focuses on producing DRC-clean layouts by enforcing rule compliance through constraints or filtering, while ignoring the generation of controllable violations. To address this gap, we propose **DRCGen**, a controllable framework for generating DRC violation patterns with cross-PDK transferability. **DRCGen** is based on a diffusion model with conditional control, generating structured control hints to encode spatial and semantic violation information for fine-tuning the model. The model can generate violation patterns of user-specified types within a target region based on a natural-language prompt, while maintaining DRC-compliance outside the region. Additionally, we incorporate few-shot learning to facilitate rapid transfer across different PDKs. Experimental results show that, compared to state-of-the-art methods, our approach achieves a 2.27 $\times$ increase in topological diversity and a 1.02 $\times$ increase in geometric diversity. Compared to Calibre LSG, we achieve 3.31 $\times$ and 1.15 $\times$ improvements in topological and geometric diversity, respectively.

1 Introduction

Layout pattern generation has long served as an essential tool in design for manufacturability (DFM) [1–3]. Specifically, layout patterns containing design rule checking (DRC) violations play an important role in both industry and academia. In industry, foundries require violation samples to verify the consistency of design-rule decks with the design rule manual (DRM) and to assess circuit behavior near rule boundaries during process design kit (PDK) optimization [4]. Furthermore, EDA tool developers use diverse violation patterns to evaluate the coverage and correctness of DRC checkers, ensuring reliable rule verification. In academia, researchers developing AI-based DRC checking [5, 6] or hotspot prediction [7, 8] methods also rely on large-scale violation samples for training. However, obtaining such data is costly and time-consuming due to the lengthy logic-to-chip design cycle. This highlights the need for automated methods for generating violation cases.

A variety of layout pattern generation methods have been developed in recent years. Early works were primarily rule-based or heuristic approaches [9–11], which generated new patterns by recombining predefined layout primitives or performing rotations and flips on existing ones. However, such data augmentation techniques produced limited pattern quantity and diversity. Similarly, industrial tools like Calibre Layout Schema Generator (LSG) rely on geometric recombination and parameter perturbation to generate boundary-case patterns, but their ability to produce complex topological variations is also constrained. With the advancement of

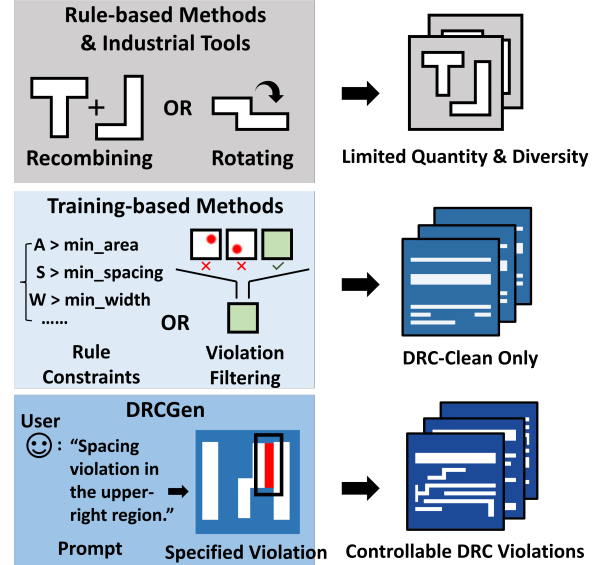


Figure 1: Comparison among traditional methods (including rule-based approaches and industrial synthetic-pattern tools), learning-based methods, and our proposed **DRCGen for DRC violation case generation.**

machine learning, pattern generation has evolved toward learning-based approaches, continuously improving as AI model architectures advance. For instance, DeePattern [12] and CUP [13] adopted autoencoder-based generators to synthesize 1D and 2D layout patterns, respectively. Both methods utilized the squish pattern [14] technique to decompose layout patterns. LayoutTransformer [15] encoded layout patterns as one-dimensional sequences containing geometric coordinates and employed a transformer model for pattern synthesis.

Following the superior performance of diffusion models in image generation [16, 17], researchers began exploring diffusion-based frameworks. DiffPattern [18] introduced a discrete diffusion model and further proposed the Deep Squish Pattern representation to compress the input tensor for improved efficiency. This line of work was later extended in ChatPattern [19], which incorporated large language model (LLM) to create a pattern generation agent. More recent studies, such as PatternPaint [20] and Vario [21], directly trained diffusion models on pixel-level layout images, enabling the understanding of complex DRC rule sets.

However, the aforementioned studies focus solely on generating DRC-clean layouts, lacking effective mechanisms to control or synthesize patterns with specific violations. These approaches either formulate the generation process as linear programming [12] or nonlinear constraint-solving [18, 19] to explicitly avoid violations, or allow models to learn design rules implicitly from data and then discard the layouts containing violations as failed cases [15, 20, 21].

The second research gap lies in cross-PDK transferability. Transferability remains a major limitation of AI-based layout generation methods. Except for PatternPaint [20], the aforementioned studies rely on large-scale layout datasets from a single technology node for model training. However, the availability of layout data is often limited, especially during the early stages of new PDK development. PatternPaint [20] achieved few-shot learning in a single technology node, but at the cost of a significantly lower success rate compared with Vario [21], which was trained with much larger datasets from the same PDK. [22] extracts shared features through attention channels, enabling fast transfer from two-dimensional patterns to simplified one-dimensional patterns.

To address these limitations, we propose **DRCGen**, a natural-language-controllable DRC violation case generator with cross-PDK transferability. Fig. 1 provides a comparative overview between **DRCGen** and previous methods. **DRCGen** is fine-tuned from the ControlNet [23], an extension of the diffusion model with conditional control capability. We encode tensors containing violation location and type information as structured control hints, enabling the model to generate specific violations within designated regions while maintaining DRC-clean layouts outside those regions. Additionally, the model is initially fine-tuned on a base PDK and then adapted to new PDKs via few-shot learning, enabling efficient transfer with minimal training data. Our contributions are as follows:

- We propose **DRCGen**, the *first* DRC violation case generator designed for industrial applications, including rule-deck development and EDA tool validation. Unlike prior methods that only produce DRC-clean layout patterns, **DRCGen** controllably introduces user-specified rule violations in designated regions via natural language prompts while maintaining DRC-clean surroundings.
- Through few-shot learning, **DRCGen** quickly adapts to new PDKs with only a few training samples, demonstrating strong transferability across technology nodes.
- Compared with Calibre LSG and KLayout, **DRCGen** achieves 3.31× higher topological diversity and 1.15× geometric-size diversity. Compared with state-of-the-art (SOTA) work, **DRCGen** achieves 2.27× higher topological diversity and 1.02× geometric-size diversity.

2 Preliminaries

2.1 Diffusion Model

Diffusion models [16] have demonstrated outstanding performance in image generation by adopting a progressive generation paradigm that consists of two Markov chains: a forward diffusion process and a reverse denoising process.

In the forward diffusion process, given an original clean sample x_0 , Gaussian noise is gradually added over T time steps:

$$q(x_t|x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}x_{t-1}, \beta_t I\right), \quad (1)$$

where $t \in [1, T]$ denotes the diffusion step and β_t controls the noise scheduling strategy. After a large number of diffusion steps, the sample x_T converges to a predefined noise distribution, typically a standard Gaussian:

$$q(x_T|x_0) \approx \mathcal{N}(0, I). \quad (2)$$

The reverse denoising process learns to invert this chain, starting from x_T and progressively removing noise to reconstruct the clean sample x_0 . A neural network $\epsilon_\theta(\cdot)$ is trained as a noise predictor to estimate the noise added at each step. Thus, the image generation process can be expressed as an iterative denoising procedure:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)), \quad (3)$$

where the mean term $\mu_\theta(x_t, t)$ depends on the predicted noise $\epsilon_\theta(x_t, t)$. The training objective is simplified to minimizing the noise prediction error:

$$\mathcal{L}_{\text{DM}} = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]. \quad (4)$$

Among various diffusion-based architectures, Stable Diffusion (SD) [24] exhibits exceptional text-to-image synthesis capability. It can be divided into three components: an encoder, a middle U-Net [25], and a decoder. The encoder, implemented as a variational autoencoder (VAE), maps input images into a latent space to reduce computational cost, where the middle performs the diffusion and denoising processes. SD also incorporates a pretrained language encoder, using text embeddings to achieve semantic control of the generation process through natural-language prompts.

2.2 ControlNet

Although Stable Diffusion is capable of producing high-quality images, its controllability remains inherently limited when relying solely on textual prompts. ControlNet [23] introduces an architecture that provides spatially localized and fine-grained conditional control. The core idea is to preserve the strong generative capability of the pretrained model while attaching a lightweight, trainable branch to process external conditional information.

Specifically, ControlNet duplicates the encoder and middle blocks of the pretrained SD and freezes all original parameters. These duplicated modules act as trainable copies and are connected to the frozen backbone through zero-initialized convolution layers, which gradually learn nonzero weights during training. This design ensures that no harmful perturbations are introduced at the early stage, allowing ControlNet to reuse the robust feature representations of the base model while efficiently learning task-specific control signals. External conditions are encoded into latent feature maps, guiding the denoising and image synthesis processes under conditional control.

In our work, ControlNet serves as the backbone of **DRCGen**, where its conditional mechanism is further extended to encode DRC violation information, enabling precise and controllable generation of violations.

3 Problem Formulation

In this section, we formalize the controllable DRC violation case generation task and the evaluation criteria. We evaluate the validity and diversity of generated cases, and further analyze the limitations of existing diversity metrics while introducing new metrics that offer improved fairness and cross-PDK generality.

Design rules are established based on process limitations at different technology nodes to ensure the intended performance and yield. As shown in Fig. 2(a), typical design rules include constraints on minimum spacing, minimum width, and minimum area. Then we define,

Definition 1 (Validity). A generated layout pattern is deemed valid if the specified region contains exactly the requested violation type and the layout remains entirely DRC-clean outside this region.

Definition 2 (Diversity). Shannon entropy is the most widely used diversity metric in prior pattern-generation studies [12, 13, 15, 18, 20]. The first entropy metric H_1 is defined as:

$$H_1 = \sum_{i,j} P(C_{x_i}, C_{y_j}) \log P(C_{x_i}, C_{y_j}), \quad (5)$$

where C_x and C_y denote the numbers of scan lines (minus one) along the x-axis and y-axis, respectively. $P(C_{x_i}, C_{y_j})$ is the probability of encountering a pattern with complexity (C_{x_i}, C_{y_j}) in the library. However, H_1 only reflects topological diversity and ignores the geometric characteristics of patterns. To incorporate geometric information, H_2 was introduced in [20]:

$$H_2 = \sum_{i,j} P(\Delta x_i, \Delta y_j) \log P(\Delta x_i, \Delta y_j), \quad (6)$$

where Δx and Δy are the vectors formed by the distances between each adjacent pair of scan lines, as shown in Fig. 2(b).

However, both H_1 and H_2 are strongly influenced by the layout size and the number of patterns within the layout. Larger layout sizes naturally contain more scan lines and polygons, leading to higher H_1 . Even with matched layout sizes, advanced-node PDKs with smaller device dimensions support more polygons, expanding geometric variation, while coarser PDKs saturate the layout more quickly. As a result, these diversity metrics are highly PDK-sensitive, complicating fair comparisons across technology nodes, such as when extending to commercial 3 nm nodes in [20].

To this end, we propose the normalized topological entropy H_1^{norm} and geometric entropy H_2^{norm} . We define n_x^{max} and n_y^{max} as the maximum numbers of scan lines allowed under the given layout size and design rule constraints. For the basic rule set, we have :

$$n_x^{max} = \left\lfloor \frac{W}{(s_{min} + w_{min})/2} \right\rfloor, n_y^{max} = \left\lfloor \frac{H}{(s_{min} + w_{min})/2} \right\rfloor, \quad (7)$$

where s_{min} and w_{min} are the minimum spacing and minimum width allowed by the design rule set. The upper bound of the topological capacity is therefore defined as:

$$N_{cap} = n_x^{max} \cdot n_y^{max}. \quad (8)$$

Accordingly, the H_1^{norm} and H_2^{norm} can be defined as:

$$H_1^{norm} = \frac{H_1}{\log(N_{cap})}, H_2^{norm} = \frac{H_2}{\log(N_{cap})}. \quad (9)$$

This normalization removes the inflation effect caused solely by having more scan lines or a richer set of DR-allowed spacing options. The resulting H_1^{norm} and H_2^{norm} measures how much of the DR-permitted topological and geometric space is actually explored by the generator, rather than the absolute count of spacing combinations. This enables fair diversity comparisons across different PDKs and layout sizes.

Based on the above evaluation metrics, the violation case generation can be formulated as follows,

Problem 1 (Violation Case Generation). *Given a set of design rules and a valid pattern library, the objective of violation case generation is to synthesize a set of **valid** patterns that (i) intentionally violate one or more specific design constraints, and (ii) maximize the **diversity**.*

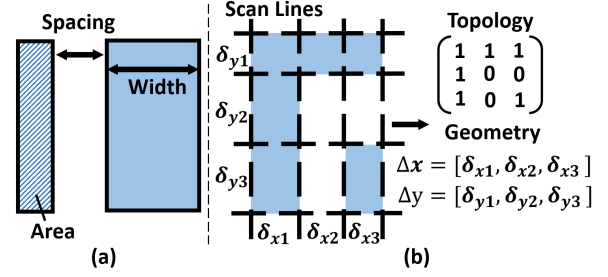


Figure 2: (a) Examples of basic DRC rules. (b) Squish pattern representation.

4 Controllable DRC Violation Case Generation

4.1 Overall Framework

The workflow of **DRCGen** is illustrated in Fig. 3. Our framework is built upon the ControlNet architecture, and a denoising post-processing step is applied at the output stage to improve the legality rate of the generated layouts. During training, the model takes three parts of inputs: layout images, multi-dimensional tensor-based control conditions (hints), and text-form prompts. The model is fine-tuned in three stages, enabling (i) generation of layout-style patterns, (ii) conditional synthesis of DRC violations, and (iii) rapid adaptability across different PDKs. The parameters updated in each fine-tuning stage are highlighted with dashed boundaries in Fig. 3, while those outside the dashed areas remain frozen to prevent catastrophic forgetting of pre-trained knowledge. Details of each fine-tuning stage will be discussed later. During inference, the user only needs to provide a natural-language prompt. **DRCGen** automatically extracts key words and generates the corresponding control hints, which jointly guide the model to synthesize targeted violation cases.

4.2 Fine-Tuning for Layout-Style Adaptation

In the first stage, we fine-tune a publicly available pretrained SD model to generate patterns that follow the visual characteristics of IC layout images. During this stage, we update all parameters in the ControlNet branch and the decoder of the SD branch. Unlocking the decoder improves the model’s ability to specialize in the target task and adapt to the visual properties of layout-style images.

Layout images are commonly stored with a black background and white foreground. Since these images contain large amounts of Manhattan geometry, the dataset exhibits a distinct visual style. In this stage, we use easily accessible open-source layout datasets that do not contain PDK information or DRC constraint labels. The model does not require control hints at this point, and an empty tensor is used as a placeholder to keep the interface consistent. After this step, **DRCGen** learns the concept of layout generation and becomes capable of synthesizing layout-style patterns.

4.3 Conditional Hint Construction

This section describes the construction of control hints, which represents the central challenge of **DRCGen**. The training data for this stage must include both the specific DRC violation type and its precise location. Such data are difficult to obtain, which is in fact one of the main motivations of **DRCGen**. To address this

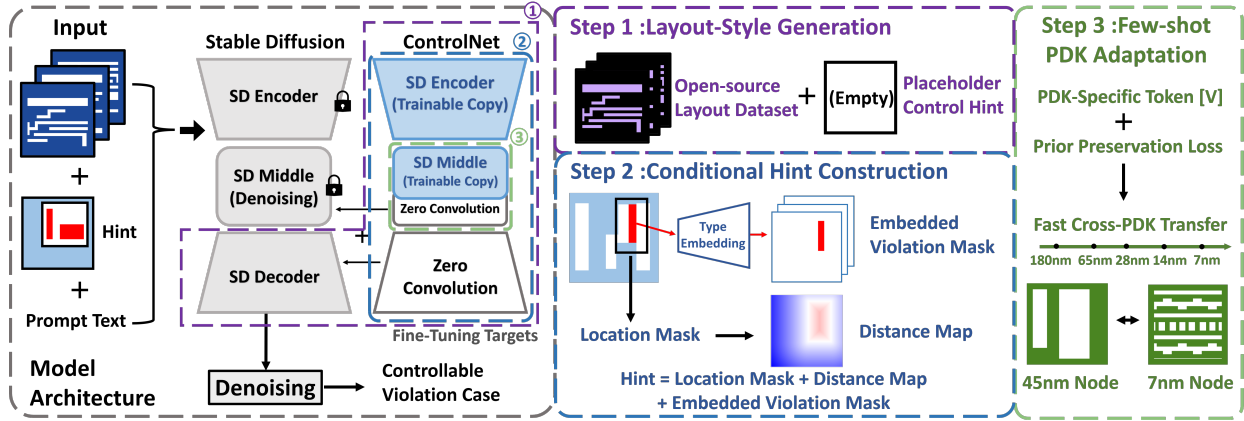


Figure 3: Overview of the *DRCGen* framework. The left panel shows the model architecture, where dashed boxes highlight the submodules fine-tuned at each step, with all other parameters kept frozen. The right panel illustrates the three fine-tuning steps: Step 1 adapts the model to layout-style generation; Step 2 constructs control hints by combining location masks, distance maps, and embedded violation masks; Step 3 enables fast cross-PDK adaptation via few-shot learning with PDK-specific tokens and prior preservation loss.

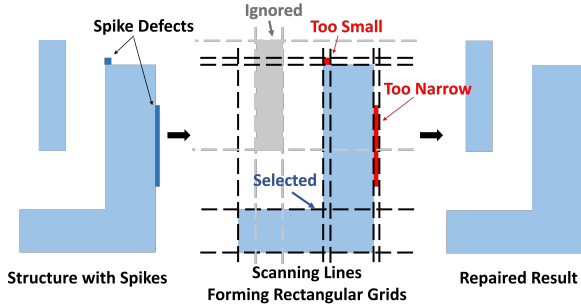


Figure 4: Illustration of our GBSS algorithm. For each connected component, the method constructs boundary-aligned scanning lines to form a rectangular grid, identifies small or narrow boundary regions as spike artifacts, and removes them to obtain a clean layout pattern.

issue, we extend the random layout generation algorithm proposed in [6]. Based on the parameters of a given PDK, we define the allowable width range of polygon patterns, then randomly sample initial positions, walking directions, and segment lengths to perform stochastic path growth within the layout area. In this work, we focus on three representative design rules on the M1 metal layer, namely minimum spacing, minimum width, and minimum area. After generating the layout pattern, we scan the layout, identify the DRC violations, and annotate them on a mask with the same size as the input image. This violation mask provides pixel-level supervision for the violation regions, and we enhance it to include additional information.

First, different violation types are represented by distinct integer values in the mask, referred to as type identifiers. For example, the values 1, 2, and 3 correspond to spacing, width, and area violations. Although these integers have numerical relationships, there is no physical meaning associated with their magnitude. Directly using this mask as a hint may mislead the model. To avoid this issue, we introduce a type embedding layer that maps each type identifier

to a high-dimensional representation, allowing the model to learn type-specific patterns in the feature space. Next, we generate an enlarged rectangular region around each violation, referred to as the location box, and construct a location mask. This mask provides contextual layout information around the violation and guides the model to focus on the relevant region. Based on the distance of each pixel to the boundary of the location box, we further construct a distance map, which assigns different weights to layout regions according to their proximity to the violation. This reinforces the model’s understanding of the violation and its surrounding context. Finally, we combine the location mask, the distance map, and the embedded violation mask into a multi-channel tensor, which serves as the control hint for ControlNet.

4.4 Grid-Based Boundary Spike Suppression

Denosing is an essential component in pixel-based layout pattern generation methods. Unlike traditional approaches that separately generate geometric topology and physical dimensions, pixel-based diffusion models cannot guarantee perfectly clean outputs. Occasionally, small geometric spikes may appear along polygon boundaries, as illustrated in Fig. 4. Such spikes rarely occur in real layout designs and violate physical design intent. They introduce additional scanning lines, artificially inflate pattern diversity, and may also trigger false positives during DRC. PatternPaint [20] proposed a template-based denosing scheme to address this issue. Building upon this idea, we develop Grid-Based Boundary Spike Suppression (GBSS), an improved template-free denosing algorithm that operates directly on connected components and their boundary-aligned scanning-line grids.

Fig. 4 illustrates the denosing process and the resulting corrected geometry. For each pattern, we first isolate its connected component to avoid interference from neighboring shapes. Within each component, we extract boundary segments and extend them into scanning lines whose intersections form a rectangular grid. The pattern is partitioned into a set of rectangles by this grid, and

any rectangle located near the boundary whose area or width falls below predefined thresholds is identified as a spike and removed.

4.5 Cross-PDK Few-Shot Adaptation

After completing ControlNet training, we further enable rapid adaptation across different PDKs by incorporating the class-specific prior preservation loss introduced in DreamBooth [26]. Specifically, we construct an instance dataset containing a small number of real layout clips from the target PDK. These samples provide the geometric characteristics and rule-specific information that the model needs to learn. In addition, we prepare a prior dataset composed of layout patterns generated by the model trained on the original PDK. The middle U-Net block in the ControlNet branch is then fine-tuned on this combined dataset. The new training objective is defined as:

$$E = E_{\text{instance}} + \lambda E_{\text{prior}}, \quad (10)$$

where E_{instance} is computed from samples in the target-PDK dataset, E_{prior} is computed from samples in the prior dataset, and λ controls the strength of prior preservation. Through the prior dataset, our fine-tuning strategy incorporates new PDK-specific geometric features while effectively preventing the model from forgetting the learned general layout style.

We also assign rare-token prompts to the instance and prior datasets to strengthen the model’s memory of each PDK. The only difference between the two PDK prompts lies in a rare token $[V]$. This token is a very low-frequency English word from the tokenizer vocabulary. It does not need any physical meaning but instead provides a clean embedding slot that binds to the target PDK data, minimizing interference from pretrained knowledge. During PDK adaptation, different PDKs can be assigned different $[V]$ tokens, which enables incremental adaptation and mitigates catastrophic forgetting. During inference, the user’s prompt is simply augmented with the corresponding PDK-specific $[V]$ token to guide the model to generate layout patterns under that PDK.

5 Experimental Results and Discussions

5.1 Data Preparation and Experimental Setup

Model setting. Our model architecture is based on ControlNet 1.0 [23] with pretrained weights from Stable Diffusion V1.5 [24].

Data preparation. For the first fine-tuning stage, we use the entire dataset from the ICCAD Contest 2012 [27]. For the second stage, we generate 10,000 layout clips of size 512 nm by 512 nm on the M1 metal layer using the 7 nm open-source PDK ASAP7 [28]. Each clip is converted into an image with a resolution of 1 pixel/nm². Among these samples, we include 2,000 spacing violations, 4,000 width violations, and 4,000 area violations. The larger proportions of width and area violations are chosen because their randomly generated instances may also contain incidental spacing violations, which we annotate together in the violation mask. For the final stage, we use 200 layout clips from a commercial 28 nm PDK.

Training setting. During training, the learning rate is set to 1e-5. The first and second fine-tuning stages take approximately 9 hours and 5 hours, while the third stage, which performs rapid PDK adaptation, requires only 10 minutes. All training and inference are performed on 4 Intel Xeon Platinum 8468 CPUs and an NVIDIA H100 80 GB GPU. In the second stage of control-hint construction,

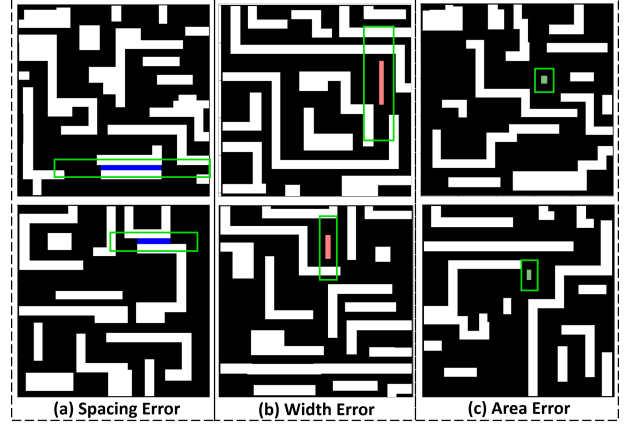


Figure 5: Generation results for three types of violation.

Table 1: Violation Case Generation Result.

Violation Type	Generated Pattern	Valid Pattern	Valid Rate
Spacing	16075	3324	20.7%
Width	17699	1524	8.6%
Area	16226	2186	13.5%
Average	-	-	14.1%

Table 2: Performance Comparison with SOTA Work.

Method	Valid Rate	H_1	H_2	Avg Runtime
DRCGen	14.1%	9.34	12.26	1.10
PatternPaint [20]	9.2%	4.11	12.01	1.02

each violation region is enlarged by a factor of 2.5 to form a location box, and overlapping boxes are merged into larger rectangles. Each type identifier is mapped to an eight-dimensional embedding, and the prompt is generated based on the violation types.

Inference. During inference, users only need to provide a text prompt. **DRCGen** automatically extracts the keywords and constructs the corresponding control hints. If the prompt contains positional descriptions such as “upper-left”, a location box is generated accordingly; otherwise, the location box is sampled randomly.

5.2 Comparison of Generation Performance

In this section, we evaluate the performance of violation case generation and compare it with state-of-the-art (SOTA) layout-generation methods. It is important to emphasize that our task fundamentally differs from prior academic studies that focus exclusively on generating DRC-clean layouts. Our validity criterion, defined in Section 3, imposes a stricter requirement. Nevertheless, we include the comparison results as a reference to illustrate the performance advantage of our **DRCGen**.

For each violation type, we randomly assign prompts with equal probability and generate 50,000 inference results in total. The number of generated samples, valid samples, and the valid rate are summarized in Table 1. The model demonstrates strong control over the area and width of foreground polygons, while its understanding of spacing in the background is relatively weaker. As a

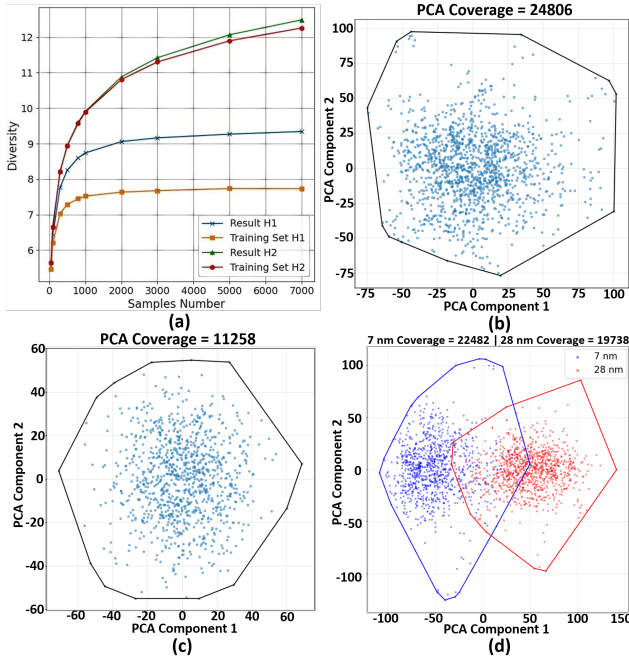


Figure 6: (a) Diversity metrics as a function of the number of generated samples. (b) PCA coverage of the generated set (24,806). (c) PCA coverage of the training set (11,257). (d) PCA comparison between the 7 nm node (blue) and the 28 nm node (red).

result, width and area violation tasks contain a number of discarded cases caused by unintended spacing errors, which leads to spacing violations exhibiting the highest validity rate among the three types. Fig. 5 presents representative generation results for the three tasks. The green boxes denote the location boxes that identify the violation regions. Within each box, the violation area is marked by a pixel-level mask, where the three colors correspond to the three violation types.

We compare our method with the SOTA PatternPaint [20] in Table 2. Although H_2 mathematically captures geometric-size diversity, the pixel-level sensitivity of Δx and Δy leads to degeneration in practical scenarios with limited generated samples. When each pattern exhibits unique Δx and Δy values, H_2 becomes solely a function of the number of generated samples N , and no longer reflects true diversity. Coarsening Δx and Δy into discrete intervals alleviates this issue to some extent. In comparison, H_1 remains the most stable and reliable metric for diversity evaluation. The results show that our method achieves a higher validity rate, along with H_1 values 2.27 $\times$ higher and H_2 values 1.02 $\times$ higher.

In addition, we compare the diversity metrics between the generated set and the training set to demonstrate how *DRCGen* enhances diversity. Fig. 6 (a) presents the diversity curves as the number of generated samples increases. We also include an equal number of samples from the training set for comparison. Results show that the generated set consistently maintains higher diversity, while H_2 continues to exhibit a significant upward trend. Furthermore, we randomly select 1,000 generated patterns and 1,000 training samples, extract the first two PCA components, and visualize the

coverage area in Fig. 6 (b) and Fig. 6 (c). PCA measures the principal directions of pixel variation and, although not directly tied to specific physical parameters, provides a proxy for the extent of exploration in the design space. As shown in the figures, *DRCGen* achieves a 2.2 $\times$ larger coverage in PCA space than the training data, indicating that it explores a substantially broader design space.

5.3 PDK Transferability Performance

We conducted the PDK transfer experiment using 200 layout clips with spacing violations at the 28 nm node, setting the loss function weight ratio λ to $\frac{1}{3}$. This means the new dataset contributes three times more weight to the loss function compared to the prior dataset. After fine-tuning, we used the model to generate 3,000 new spacing violation samples. The results show that we successfully obtained 254 valid violation cases under the 28 nm design rules, achieving an validity rate of 8.5%. We performed PCA coverage area analysis on the newly generated and prior datasets, as shown in Fig. 6 (d), where the distribution of the results clearly indicates a successful transfer. The H_1 value for this dataset is 7.80, and the H_1^{norm} for the two PDKs are 3.21 (7 nm) and 3.50 (28 nm), remaining at similar levels. This demonstrates that normalization partially alleviates the impact of the PDK itself on diversity evaluation.

5.4 Comparison with Industrial Tools

Industrial standard software such as Calibre LSG employs tile-based geometry generation to construct layout patterns. The tool discretizes the layout space into uniform grid cells and fills these cells according to design rule constraints. The open-source tool KLayout can implement the same functionality through scripting. For reproducibility, we use KLayout to generate the same number of violation and boundary-case layout clips at the 7 nm node as the valid outputs produced by *DRCGen*. The layout size, the number of shapes within each clip, and the ranges of geometric parameters are all kept consistent with the training data of *DRCGen*. After generation, we obtain $H_1 = 2.82$, $H_2 = 10.62$. In comparison, our method achieves 3.31 $\times$ higher topological diversity H_1 and 1.15 $\times$ higher geometric diversity H_2 . This clearly highlights the limitations of industrial tools in terms of topological diversity.

6 Conclusion

In this paper, we present *DRCGen*, a controllable DRC violation generator for rule-deck and EDA tool validation, with support for rapid cross-PDK transfer. We encode violation information at both spatial and semantic levels to construct control hints and fine-tune the model based on ControlNet. This approach enables the generation of specified DRC violations in targeted regions while ensuring DRC-clean behavior outside these regions. Additionally, we leverage few-shot learning to achieve rapid transfer across PDKs, making it applicable to early-stage PDK development or scenarios with insufficient data. Experimental results show that, compared to SOTA method, our approach achieves a 2.27 $\times$ and a 1.02 $\times$ increase in topological and geometric diversity, and outperforms Calibre LSG with 3.31 $\times$ and 1.15 $\times$ improvements in topological and geometric diversity, respectively. Future work will extend support to generating cases for additional violation types.

References

- [1] Jih-Rong Gao, Xiaoqing Xu, Bei Yu, and David Z Pan. 2014. MOSAIC: Mask optimizing solution with process window aware inverse correction. In *Proceedings of the 51st Annual Design Automation Conference (DAC)*. 1–6.
- [2] Bentian Jiang, Hang Zhang, Jinglei Yang, and Evangeline FY Young. 2019. A fast machine learning-based mask printability predictor for OPC acceleration. In *Proceedings of the 24th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 412–419.
- [3] Su Zheng, Yuzhe Ma, Bei Yu, and Martin DF Wong. 2024. EMOGen: Enhancing mask optimization via pattern generation. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [4] Helen Li, Elain Zou, Robben Lee, Sid Hong, Square Liu, JinYan Wang, Chunshan Du, Recco Zhang, Kareem Madkour, Hussein Ali, et al. 2016. Design space exploration for early identification of yield limiting patterns. In *Design-Process-Technology Co-optimization for Manufacturability X*. 285–292.
- [5] Luis Francisco, Tanmay Lagare, Arpit Jain, Somal Chaudhary, Madhura Kulkarni, Divya Sardana, W Rhett Davis, and Paul Franzon. 2020. Design rule checking with a CNN based feature extractor. In *Proceedings of the ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*. 9–14.
- [6] Shunjie Chang, Youran Wu, Jianli Chen, Jun Yu, and Kun Wang. 2025. 3D DRC: Design Rule Checking for 3D IC with U-Net-based Non-Manhattan Optimization. In *Proceedings of the 44th International Conference on Computer-Aided Design (ICCAD)*.
- [7] Weihai Lu and Hong Cai Chen. 2025. MAGNet: A Multi-Scale Attention-Guided Graph Fusion Network for DRC Violation Detection. *arXiv preprint arXiv:2506.07126* (2025).
- [8] Yuyang Chen, Yiwen Wu, Jingya Wang, Tao Wu, Xuming He, Jingyi Yu, and Hao Geng. 2024. LLM-HD: Layout language model for hotspot detection with GDS semantic encoding. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [9] Gaurav Rajavendra Reddy, Constantinos Xanthopoulos, and Yiorgos Makris. 2018. Enhanced hotspot detection through synthetic pattern generation and design of experiments. In *Proceedings of the IEEE 36th VLSI Test Symposium (VTS)*. 1–6.
- [10] Gaurav Rajavendra Reddy, Kareem Madkour, and Yiorgos Makris. 2019. Machine learning-based hotspot detection: Fallacies, pitfalls and marching orders. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8.
- [11] Wei Ye, Yibo Lin, Meng Li, Qiang Liu, and David Z Pan. 2019. LithoROC: lithography hotspot detection with explicit ROC optimization. In *Proceedings of the 24th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 292–298.
- [12] Haoyu Yang, Piyush Pathak, Frank Gennari, Ya-Chieh Lai, and Bei Yu. 2019. DeePattern: Layout pattern generation with transforming convolutional auto-encoder. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*. 1–6.
- [13] Xiaopeng Zhang, James Shiely, and Evangeline FY Young. 2020. Layout pattern generation and legalization with generative learning models. In *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*. 1–9.
- [14] F. E. Gennari and Y.-C. Lai. 2014. Topology design using squish patterns. U.S. Patent 8,832,621. Issued Sep. 9, 2014.
- [15] Liangjian Wen, Yi Zhu, Lei Ye, Guojin Chen, Bei Yu, Jianzhuang Liu, and Chunjing Xu. 2022. Layouttransformer: Generating layout patterns with transformer via sequential pattern modeling. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–9.
- [16] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.
- [17] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502* (2020).
- [18] Zixiao Wang, Yunheng Shen, Wenqian Zhao, Yang Bai, Guojin Chen, Farzan Farnia, and Bei Yu. 2023. Diffpattern: Layout pattern generation via discrete diffusion. In *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [19] Zixiao Wang, Yunheng Shen, Xufeng Yao, Wenqian Zhao, Yang Bai, Farzan Farnia, and Bei Yu. 2024. Chatpattern: Layout pattern customization via natural language. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [20] Guanglei Zhou, Bhargav Korrapati, Gaurav Rajavendra Reddy, Chen-Chia Chang, Jingyu Pan, Jiang Hu, Yiran Chen, and Dipto G Thakurta. 2025. Patternpaint: Practical layout pattern generation using diffusion-based inpainting. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*. 1–7.
- [21] Guanglei Zhou, Bhargav Korrapati, Gaurav Rajavendra Reddy, Junyao Zhang, Yiran Chen, and Dipto G Thakurta. 2025. Vario: enhance pattern diversity with diffusion model. In *DTCO and Computational Patterning IV*. 19–25.
- [22] Xiaopeng Zhang, Haoyu Yang, and Evangeline FY Young. 2021. Attentional transfer is all you need: Technology-aware layout pattern generation. In *Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC)*. 169–174.
- [23] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2023. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 3836–3847.
- [24] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10684–10695.
- [25] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *Proceedings of the Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. 234–241.
- [26] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. 2023. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 22500–22510.
- [27] J Andres Torres. 2012. ICCAD-2012 CAD contest in fuzzy pattern matching for physical verification and benchmark suite. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*. 349–350.
- [28] Lawrence T Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandrasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm finFET predictive process design kit. *Microelectronics Journal* 53 (2016), 105–115.